Week 2 Module – Programming Practice (3rd Year)
Date: 22/06/2026

Language Used: JAVA

Pdf follows: handwritten code with logic and followed by the respective outputs.

Submission details:---

Name : Sakshi das

Roll number : 2404921520194

Task completed on : 22/06/2026 – Sunday

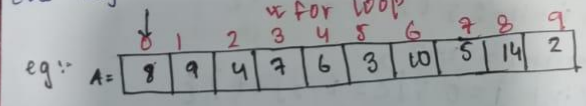
Week 2 Module.

concept: searching and sorting :-

Types of search :
i) Linear search.
ii) Binary search.

Linear search :— * To performing search - elements must be unique.
* The value that we are searching is referred as the key element.

what happens in linear search is that, search of key takes place linearly (i.e. checking every element one by one). ⟶ This can be done using * for loop * while loop.



eg :-
⇒ size = 10
⇒ length = 10.

say : key = 5 ⟶ successful
* key = 12 ⟶ unsuccessful.

```
for (int i=0; i<length; i++) {
    if (key == A[i]) {
        System.out.println ("search was
                             successful");
        return i;
    }
    else {
        System.out.println ("search was unsuccessful");
        return -1;
    }
}
```

A[i] ⇒ Array of index i.

Additional practice :—
* Time complexity : Best case : O(1)
worst case : O(n)

Imp note (i) Possibility of searching an element again is higher.

Then we can use the concept of Transposition.

i.e. swapping key one step forward so that, again when searched — no of steps REDUCES.

code becomes :—

```
for (int i = 0 ; i < length ; i++) {
    if (key == A[i]) {
        swap (A[i] , A[i-1]);
        return i-1 ;
    }
}.
```

↓ key diff b/w these two :— in "Transposition", there is slow reduction in time taken for searching.

in "Move to Head", there is a certain reduction in time taken for searching. ↑

initially Array was :

8	9	4	7	6	3	10	5	14	2

after one interation it becomes ;

8	9	4	7	6	3	5	10	14	2

ii) Another improving can be done, is that — when key element is found, move the key to front or Head ⟶ MOVE TO FRONT / HEAD.

code then becomes :—

```
for (int i = 0 ; i < length ; i++) {
    if (key == A[i]) {
        swap (A[i], A[0]);
        return 0;
    }
}.
```

Q1) FRAUD TRANSACTION SYSTEM :—

Bank → stores Transaction IDs → not sorted

needs to check suspicious Transaction IDs exist or not. ← Team

eg :— N = 5 .... // 5 Transaction IDs exist.

[1201, 1305, 1450, 1408, 1502] .... // storing it in an Array.

1450 .... // security teams says to check TID 1450".

when found → FRAUD TRANSACTION FOUND.

never not → " " NOT FOUND.

ARRAY syntax (we know) → int arr[] = new int [size];

(datatype) arrayName [] = ...

Algo :—

1. input N → Reading no. of IDs.
2. creating Array of size N.
3. Taking i/p for all IDs & store in array.
4. Read suspicious ID.
5. Check whether found/not.
6. Once found print → "print statement".
similarly not found → " " ....

~~God B :—~~

putoti

(Key concept derived from the code practiced) :—

⇒ we didn't use else statement, because writing else might hamper the result, →

eg :- | 1 | 2 | 3 | 4 | 5 |

suspicious : 3

loop will check

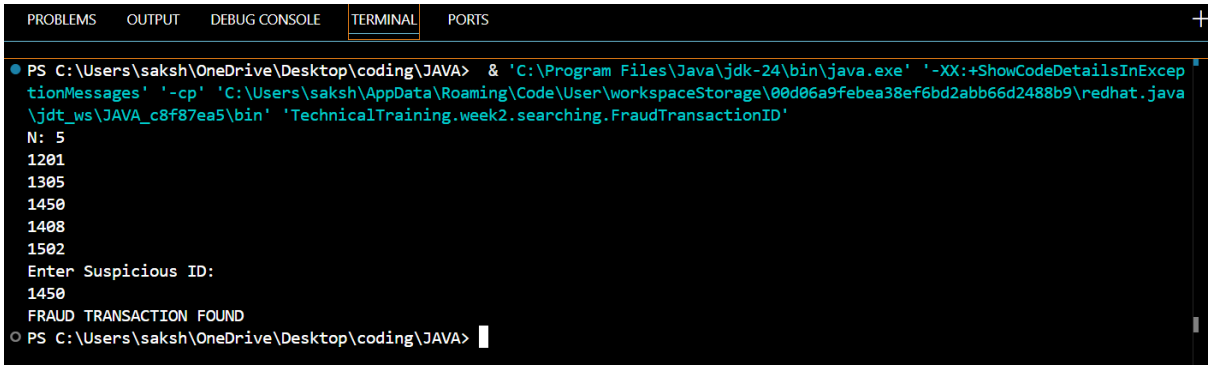
1 == 3 → false
2 == 3 → false
3 == 3 ⇒ true
4 == 3 → false
5 == 3 → false

} the result will print element not found because of else statement even though the element is present

Code :—

```
package TechnicalTraining.week2.searching;
import java.util.Scanner;
public class FraudTransactionID{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("N: ");
        int N = sc.nextInt(); //takes input of N no. of IDs. N=5.   ← from user.
        int tIDs[] = new int[N] //creating array
        for(int i=0; i<N; i++) {
            tIDs[i] = sc.nextInt(); //taking user i/p for IDs.
        }
        System.out.print("Enter suspicious ID: ");
        int suspicious = sc.nextInt(); //taking i/p for
                                       suspicious ID.
        boolean found = false; //initializing found to be zero.

        //(searching now)
        for(int i=0; i<N; i++) {
            if(tIDs[i] == suspicious) {
                found = true;
                break; //it is optional but efficient practice.
            } //we didn't use else statement because the result
        }
        if(found == true) {
            System.out.println("FRAUD TRANSACTION FOUND");
        }
        else {
            System.out.println("FRAUD TRANSACTION NOT FOUND");
        }
        sc.close();
    }
}
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\saksh\AppData\Roaming\Code\User\workspaceStorage\00d06a9febea38ef6bd2abb66d2488b9\redhat.java\jdt_ws\JAVA_c8f87ea5\bin' 'TechnicalTraining.week2.searching.FraudTransactionID'
N: 5
1201
1305
1450
1408
1502
Enter Suspicious ID:
1450
FRAUD TRANSACTION FOUND
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA>

2) Binary search :- "Divide & conquer" order

→ Data must be in sorted order.

eg :-

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
3	6	8	12	14	17	25	29	31	36	42	47	53	55	62

low(l)	high(h)	mid = $\frac{l+h}{2}$	(key = 42)
1	15	8	42 < 47
(8+1) = 9	15	12	∴ checking in 8 – 12
9	(high-1) = 11	10	36 < 42
11	11	11 → element present	

no. of comparison = 4.

* Q2). Server log search (Binary search):—

Sorted IDs ~~exists~~ given → Engineers needs to quickly check if ID exist.

method & Approach

eg i/p :- 7
1001 1005 1010 1015 1020 1030 1040
1015

target = 1015

	0	1	2	3 (mid)	4	5	6 (n-1)
Array :-	1001	1005	1010	1015	1020	1030	1040

1) Setting low = 0 high = 6.
mid = (low + high) / 2. = 3

∵ mid == key. → element exist.

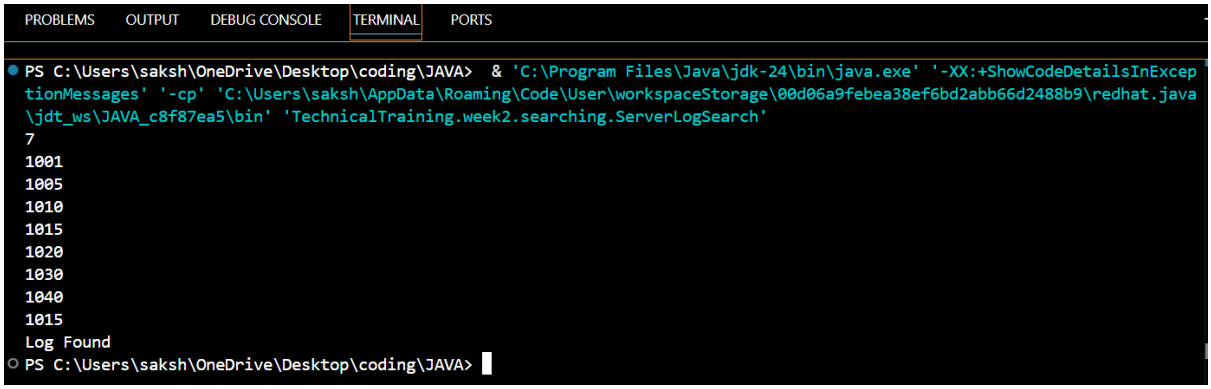
another practice. key = 1030.

low = 0
high = 6
mid = 0+6/2 = 3
1015 < 1030

→ low = mid+1 = 4
high = 6
mid = 4+6/2 = 5
1030 = 1030 → element found.

Code :-

```
import java.util.Scanner;
public class ServerLogSearch {
    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        int N= sc.nextInt();
        int logs[] = new int[N];
        for(int i=0; i<N; i++){
            logs[i] = sc.nextInt();
        }
        int target=sc.nextInt();
        int low =0;
        int high = N-1;
        ~~int mid = (low+high)/2;~~ boolean found=false;
        while (low<=high) {
            int mid =(low+high)/2;
            if (logs[mid] == target) {
                found = true;
                break;
            }
            elseif(target > logs[mid]) {
                ~~found = true;~~
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        if(found) {
            System.out.println("Log Found");
        } else {
    } } System.out.println("Log Not Found");
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\saksh\AppData\Roaming\Code\User\workspaceStorage\00d06a9febea38ef6bd2abb66d2488b9\redhat.java\jdt_ws\JAVA_c8f87ea5\bin' 'TechnicalTraining.week2.searching.ServerLogSearch'
7
1001
1005
1010
1015
1020
1030
1040
1015
Log Found
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA>

B) Sorting : Bubble sort → Repeatedly compares adjacent elements and swaps them if they are in the wrong order

eg :- 45000 32000

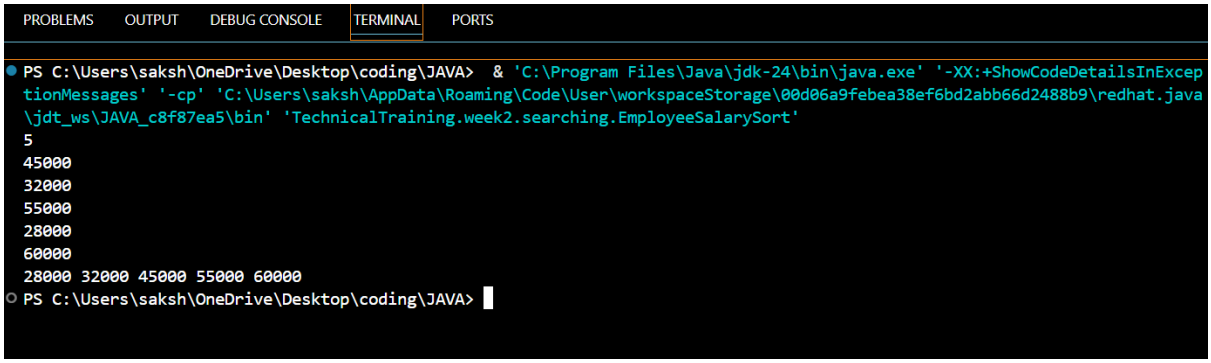
since, 45000 > 32000 ⇒ swap ⇒ ~~45000~~ 32000 45000

Algo :- i/p → Array → Read Array → start comparing, left > right ⇒ swap

(Q3) { }

code :-

```
import java.util.Scanner;
public class EmployeeSalarySort {
    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        int n = sc.nextInt();
        int salaries [] = new int[n];
        for (int i=0; i<n ; i++) {
          salaries [i] =sc.nextInt();
        }
        for (int i=0 ; i<n-1 ; i++) {
          for (int j= 0 ; j<n-1 ; j++) {
            if (salaries [j] > salaries [j+1]) {
                int temp = salaries[j] ;
                salaries[j] = salaries [j+1];
                salaries [j+1] = temp ;
            }
          }
        }
        for(int i=0; i<n ; i++) {
          System.out.print (salaries [i]+ "  ");
        }
        sc.close();
    }
}
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\saksh\AppData\Roaming\Code\User\workspaceStorage\00d06a9febea38ef6bd2abb66d2488b9\redhat.java\jdt_ws\JAVA_c8f87ea5\bin' 'TechnicalTraining.week2.searching.EmployeeSalarySort'
5
45000
32000
55000
28000
60000
28000 32000 45000 55000 60000
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA>

4) Insertion sort.
"new data comes continuously and must be inserted into the current position".

i.e. 250 120 320 180 100

↓

120 250 320 180 100

↓

120 250 320 180 100

↓

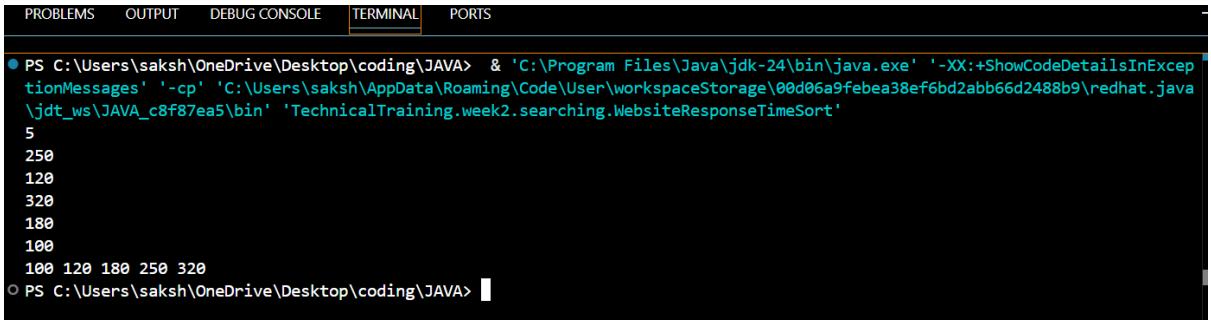
120 180 250 320 100

↓

100 120 180 250 320

code :-

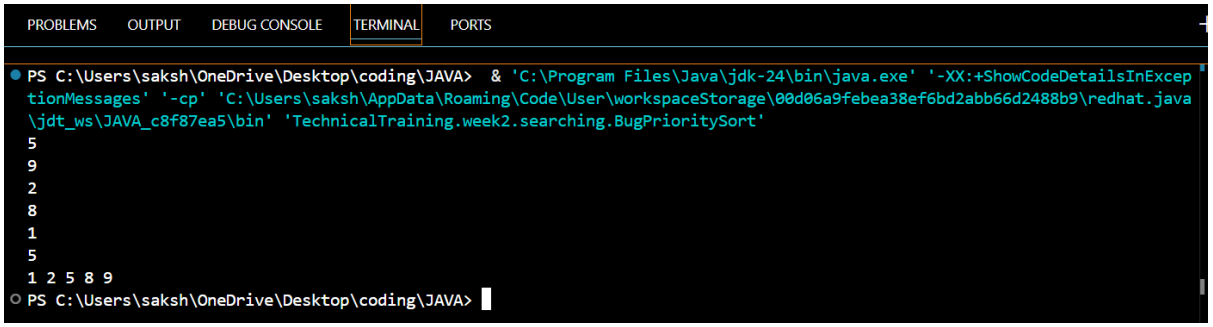
```
import java.util.Scanner;
public class WebsiteResponseTimeSort{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N=sc.nextInt();
        int responseTime[] = new int[N];
        for(int i=0; i<N; i++){
            responseTime[i]=sc.nextInt();
        }
        for(int i=1; i<N; i++){
            int key= responseTime[i];
            int j = i-1;
            while(j>=0 && responseTime[j]>key){
                responseTime[j+1]=responseTime[j];
                j=j-1;
            }
            for(int i=0; i<N; i++){
                System.out.print(responseTime[i]+" ");
            }
        }
        sc.close();
    }
}
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\saksh\AppData\Roaming\Code\User\workspaceStorage\00d06a9febea38ef6bd2abb66d2488b9\redhat.java\jdt_ws\JAVA_c8f87ea5\bin' 'TechnicalTraining.week2.searching.WebsiteResponseTimeSort'
5
250
120
320
180
100
100 120 180 250 320
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA>

Selection sort

code :-

```
import java.util.Scanner;
public class BugPriority Sort {
    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        int n = sc.nextInt();
        int[] priority = new int[n];
        for (int i=0; i<n; i++) {
            priority[i] = sc.nextInt();
        }
        for (int i=0; i<n-1; i++) {
            int minIndex = i;
            for (int j= i+1; j<n; j++) {
                if (priority[j] < priority[minIndex]) {
                    minIndex = j;
                }
            }
            int tem = priority[i];
            priority[i] = priority[minIndex];
            priority[minIndex] = temp;
        }
        for (int i=0; i<n; i++) {
            System.out.print (priority[i] + " ");
        }
        sc.close();
    }
}
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\saksh\AppData\Roaming\Code\User\workspaceStorage\00d06a9febea38ef6bd2abb66d2488b9\redhat.java\jdt_ws\JAVA_c8f87ea5\bin' 'TechnicalTraining.week2.searching.BugPrioritySort'
5
9
2
8
1
5
1 2 5 8 9
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA>